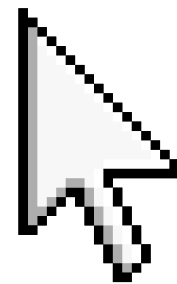


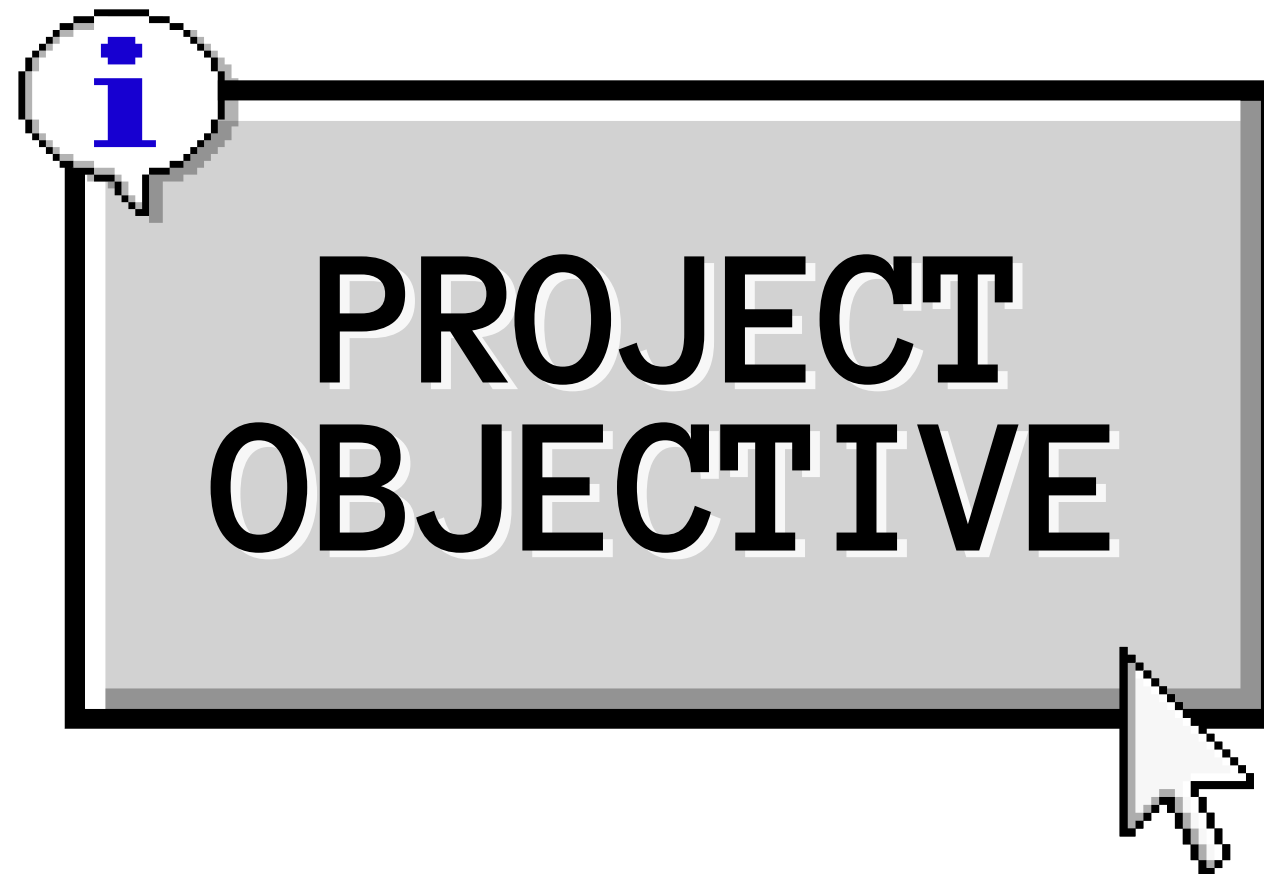
RISC-V PROCESSOR

DRDO SUMMER INTERNSHIP PROJECT



Start



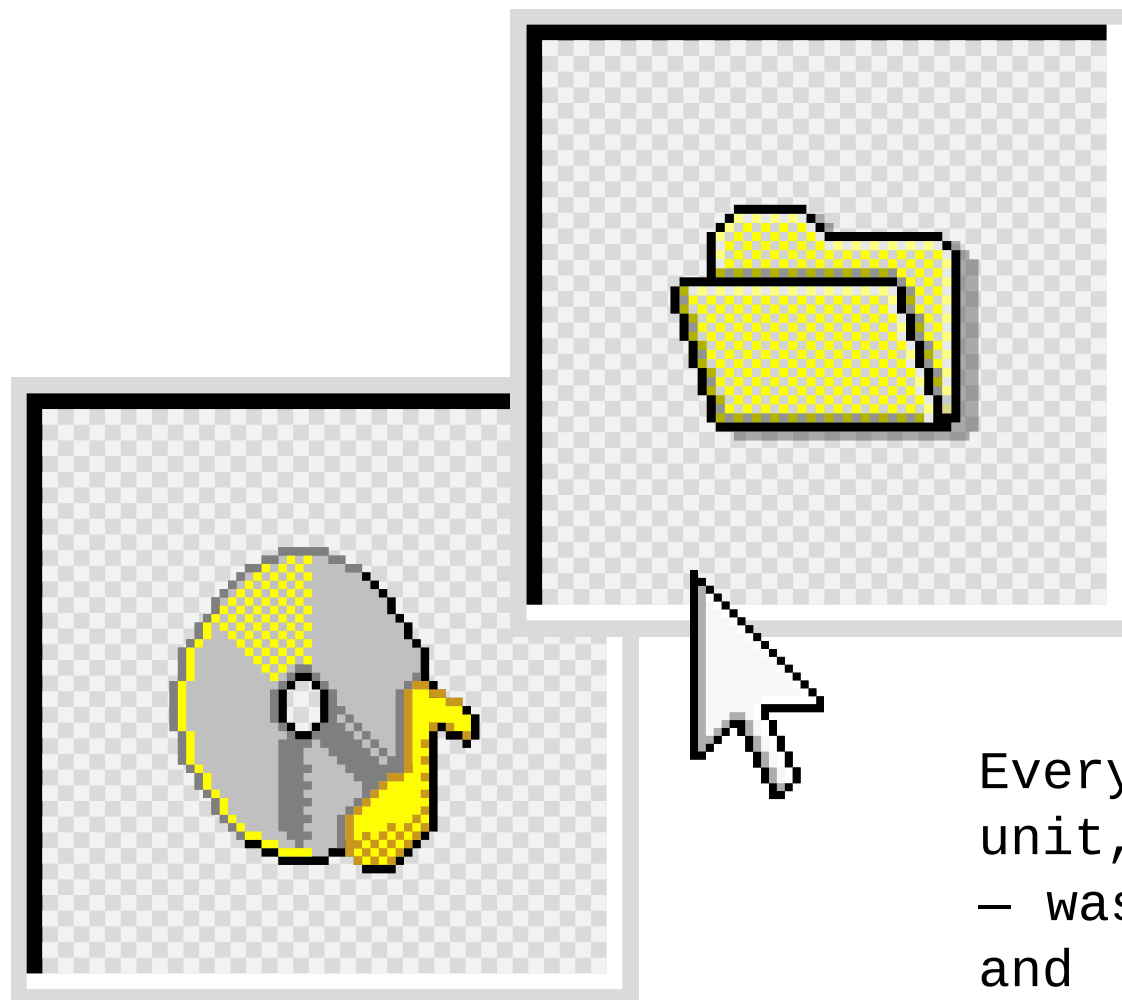


- Design and implement a functional 32-bit RISC-V single-cycle processor in Verilog, then verify its instruction flow and datapath using simulation
- Extend it into 5 stage pipelining with hazard handling



- RISC-V is an open ISA with a clean, modular instruction set
- It makes the processor datapath easy to understand, extend and implement in RTL.

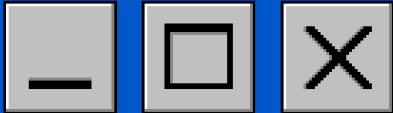

WHY RISC-V?

WHAT WAS ACTUALLY BUILT?

Every combinational and stateful datapath block – ALU, control unit, register file, immediate generator, instruction/data memory – was written once and reused unchanged by both the single-cycle and pipelined top-level modules, with the pipeline adding only pipeline registers, a hazard unit, and a forwarding unit on top.





FULL RISC-V BASE ISA COVERAGE

R-Type

ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND

I-Type ALU

ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI

Loads

LB, LH, LW, LBU, LHU (correct sign/zero extension)

Stores

SB, SH, SW

Branches

BEQ, BNE, BLT, BGE, BLTU, BGEU

Upper Immediate

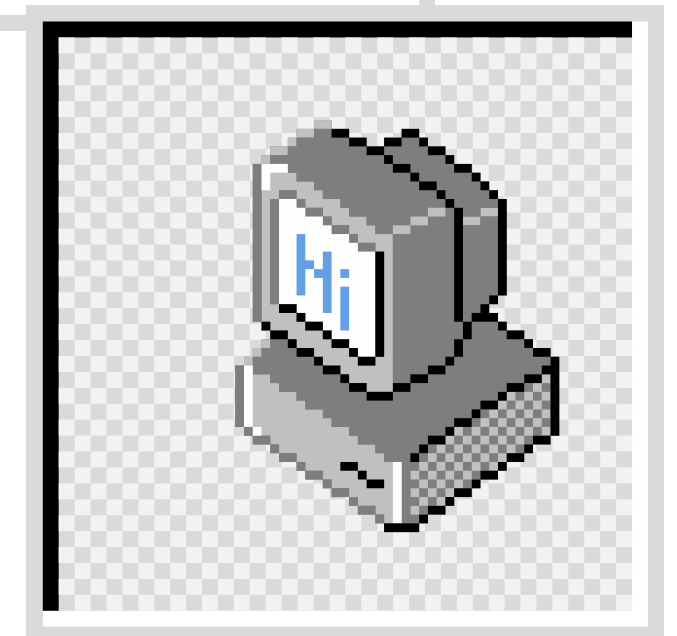
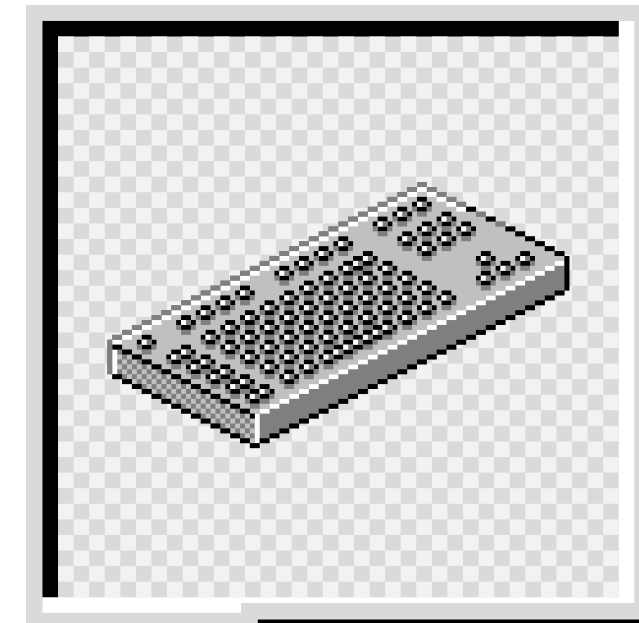
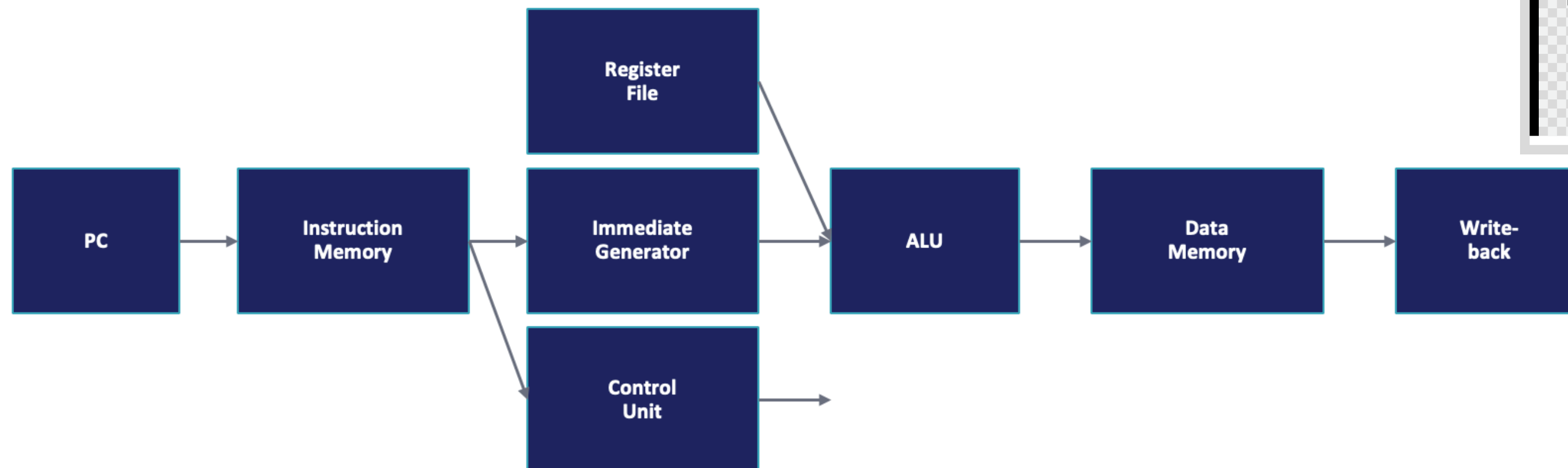
LUI, AUIPC

Jumps

JAL, JALR



SINGLE CYCLE ARCHITECTURE



WHY MOVE BEYOND A SINGLE CYCLE?

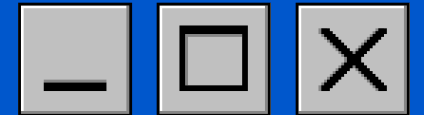
Single-Cycle

- Clock period must accommodate the slowest instruction (typically a load: fetch → decode → address calc → memory access → writeback, all in one cycle)
- A simple ADDI pays the same cycle cost as the slowest load, even though its actual logic path is far shorter.
- Hardware sits idle for most of each cycle for most instructions.

Pipelined Model

- Split execution into 5 shorter stages: Fetch, Decode, Execute, Memory, Writeback.
- Overlap stages across instructions – while one instruction executes, the next is decoding, and the one after that is fetching.
- Ideal throughput: close to one instruction completed per cycle, at a much shorter clock period.
- Cost: overlapping instructions can now interfere with each other – hazards.





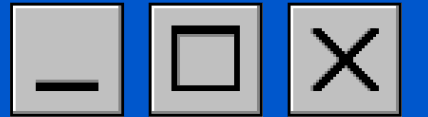
Opcode and function fields are decoded into control signals for ALU operation, register write, memory access and branch selection.



CONTROL UNIT

Control is combinational in the single-cycle design.



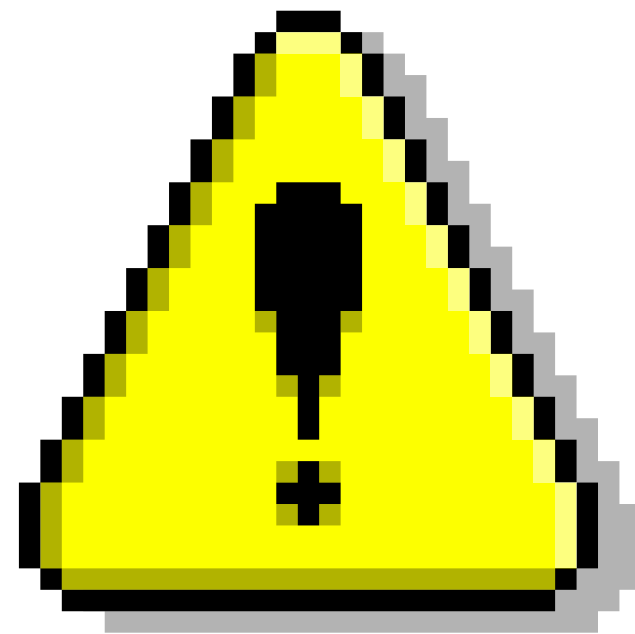


A register file provides general-purpose registers, while instruction memory and data memory provide the storage path required by the CPU.



MEMORY & REGISTERS





INSTRUCTION MEMORY

Instructions are stored in instruction memory and fetched using the program counter. The PC advances to the next instruction unless control flow changes it.

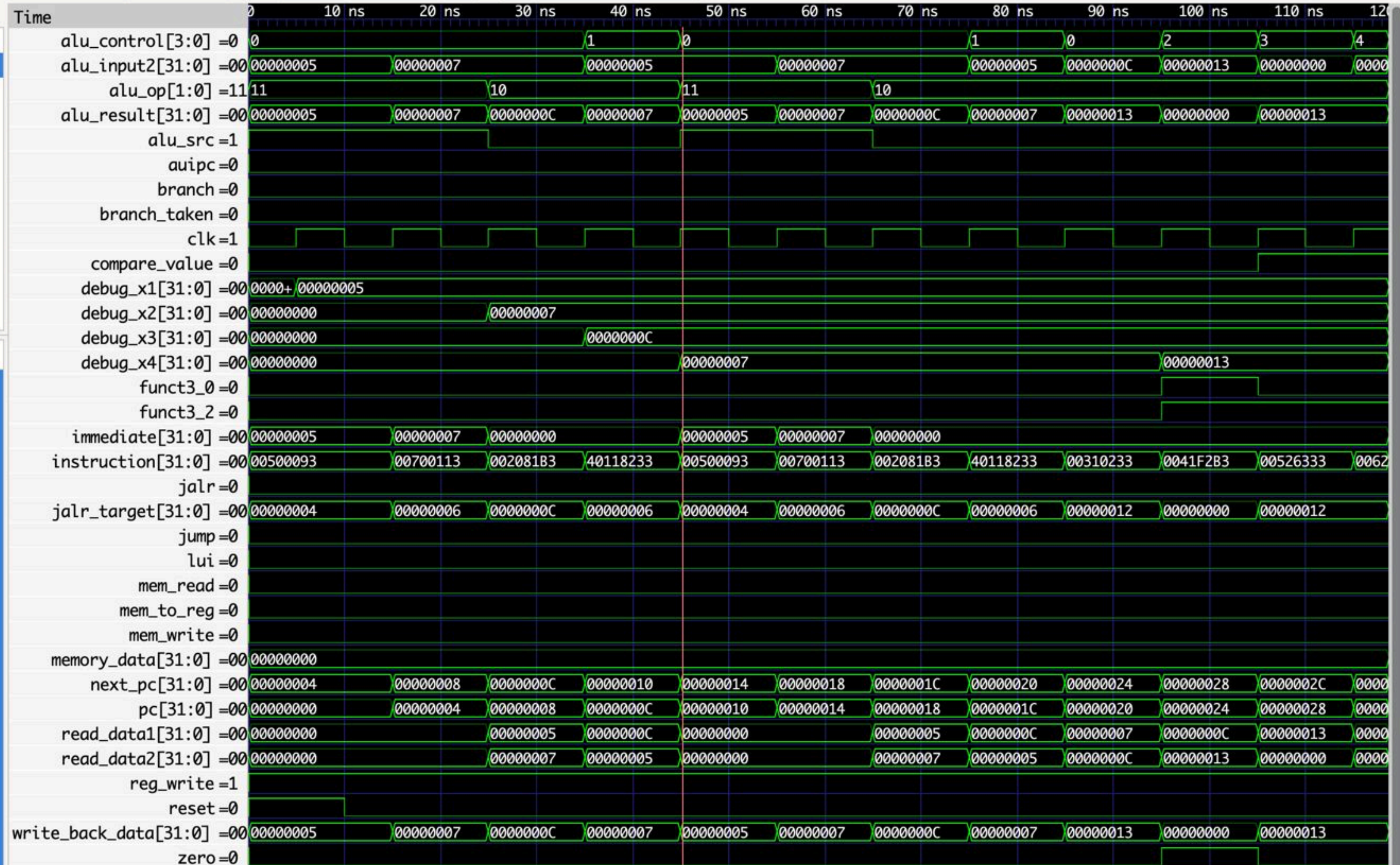


▼ SST

▼  single_cycle_cpu_tb



Type	Signals
wire	debug_x3[31:0]
wire	debug_x4[31:0]
wire	funct3_0
wire	funct3_2
wire	immediate[31:0]
wire	instruction[31:0]
wire	jalr
wire	jalr_target[31:0]
wire	jump
wire	lui
wire	mem_read
wire	mem_to_reg
wire	mem_write
wire	memory_data[31:0]
wire	next_pc[31:0]
wire	pc[31:0]
wire	read_data1[31:0]
wire	read_data2[31:0]
wire	reg_write
wire	reset
wire	write_back_data[31:0]
wire	zero

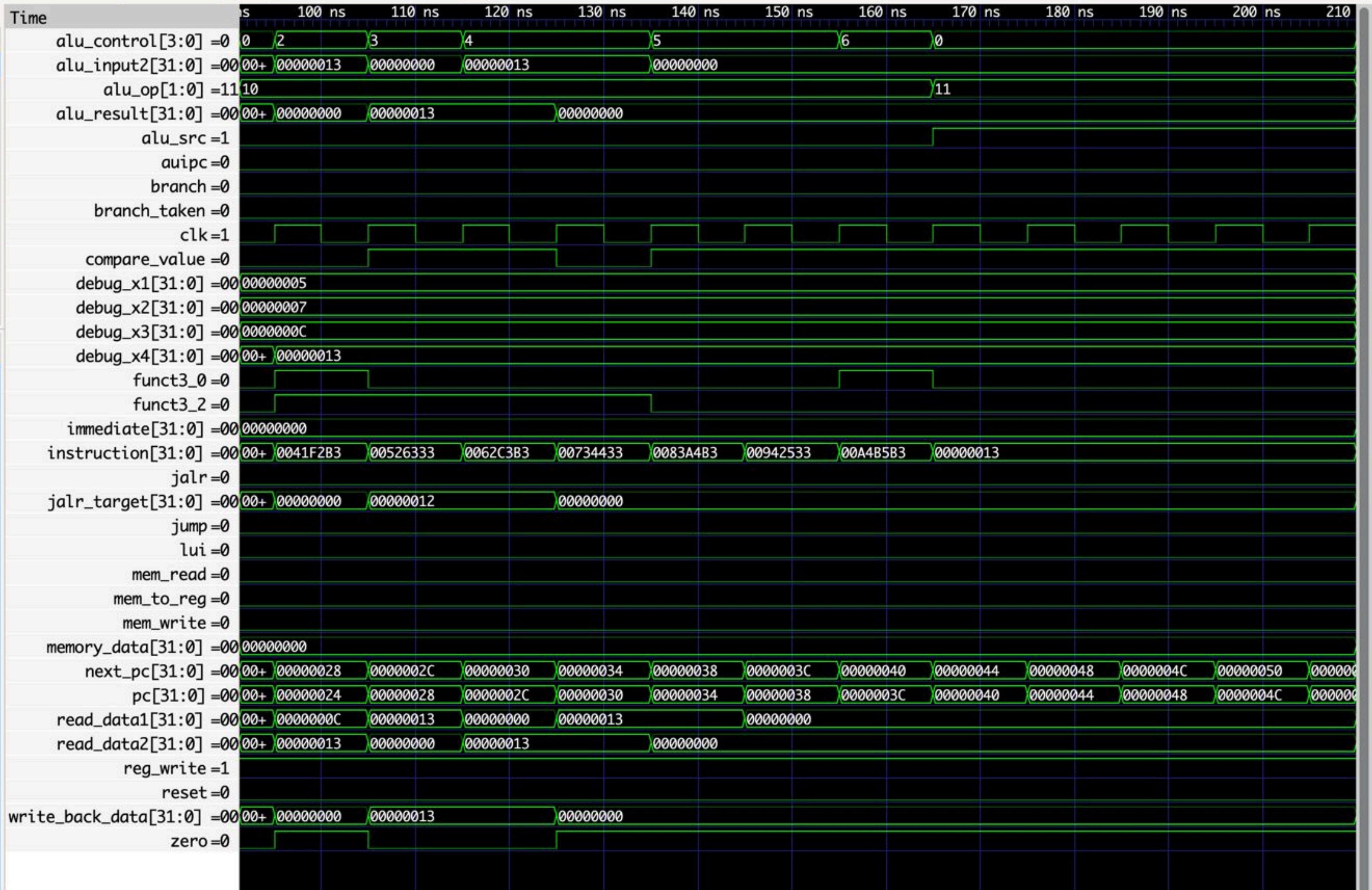


SST

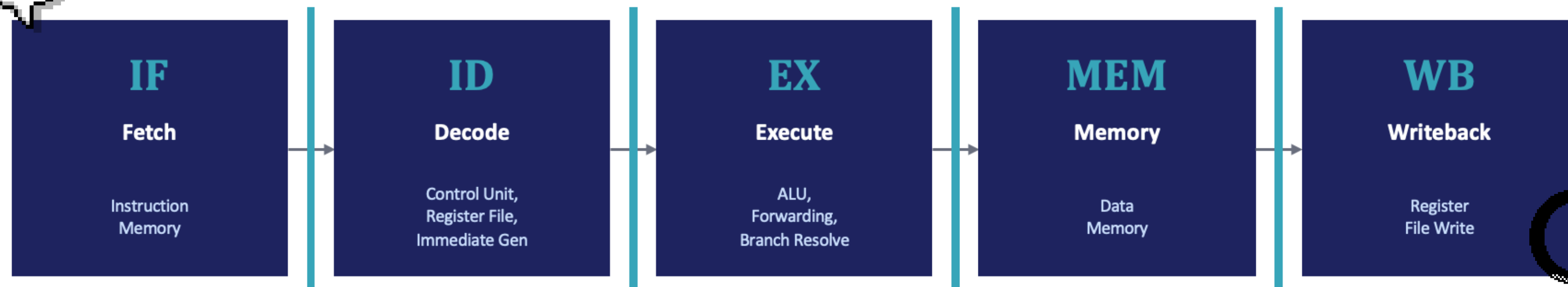
single_cycle_cpu_tb

uut

Type	Signals
wire	debug_x3[31:0]
wire	debug_x4[31:0]
wire	funct3_0
wire	funct3_2
wire	immediate[31:0]
wire	instruction[31:0]
wire	jalr
wire	jalr_target[31:0]
wire	jump
wire	lui
wire	mem_read
wire	mem_to_reg
wire	mem_write
wire	memory_data[31:0]
wire	next_pc[31:0]
wire	pc[31:0]
wire	read_data1[31:0]
wire	read_data2[31:0]
wire	reg_write
wire	reset
wire	write_back_data[31:0]
wire	zero



5-STAGE PIPELINE: IF → ID → EX → MEM → WB



Each combinational block (ALU, control unit, register file, memory) is instantiated once and lives permanently in its own stage — exactly the same modules as the single-cycle core, just wrapped with pipeline registers, a hazard unit, and a forwarding unit.



SST

pipelined_cpu_tb

uut

alu_control_inst

alu_inst

control_unit_inst

data_memory_inst

ex_mem_reg_inst

forwarding_unit_inst

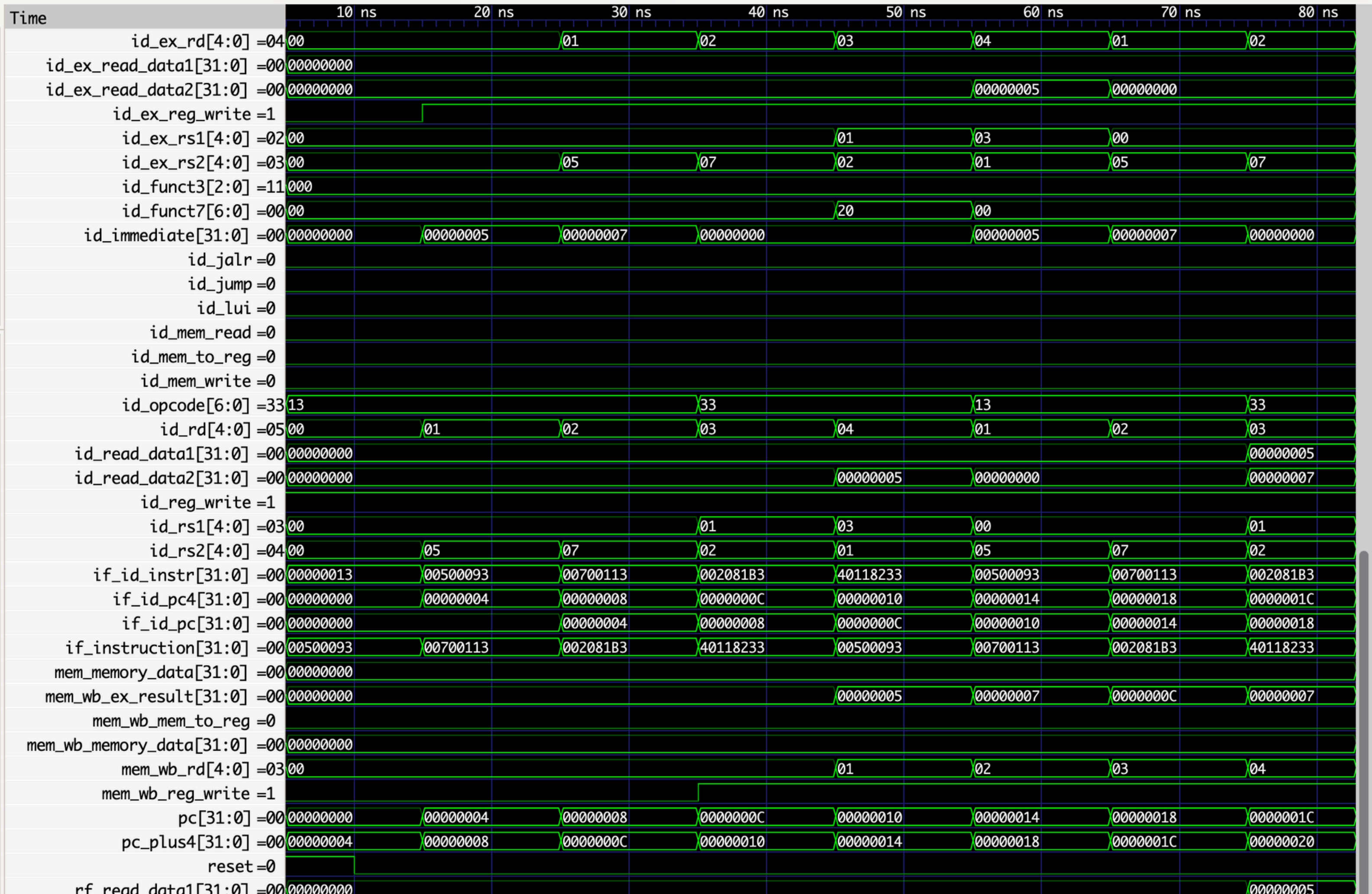
hazard_unit_inst

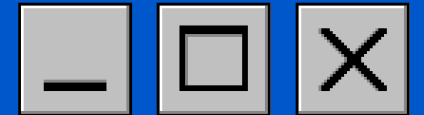
id_ex_reg_inst

if_id_reg_inst

immediate_generator_inst

Type	Signals
wire	id_read_data1[31:0]
wire	id_read_data2[31:0]
wire	id_reg_write
wire	id_rs1[4:0]
wire	id_rs2[4:0]
wire	if_id_instr[31:0]
wire	if_id_pc4[31:0]
wire	if_id_pc[31:0]
wire	if_instruction[31:0]
wire	mem_memory_data[31:0]
wire	mem_wb_ex_result[31:0]
wire	mem_wb_mem_to_reg
wire	mem_wb_memory_data[31:0]
wire	mem_wb_rd[4:0]
wire	mem_wb_reg_write
reg	pc[31:0]
wire	pc_plus4[31:0]
wire	reset
wire	rf_read_data1[31:0]
wire	rf_read_data2[31:0]
wire	stall
wire	wb_write_back_data[31:0]

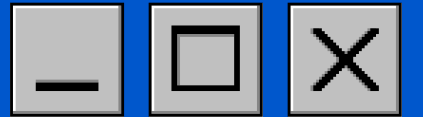




VERIFICATION STRATEGY

- **Per-Module Testbenches** : Every ALU, control unit, register file, and memory block verified independently before integration.
- **Full-ISA Stress Programs** : Hand-assembled test programs exercising every instruction class – shifts, byte/half loads, all branch types, JALR.
- **Targeted Hazard Tests** : Purpose-built dependency chains (distance 1/2/3, load-use, taken/not-taken branches) to force every hazard path.
- **Waveform-Level Proof** : GTKWave traces cross-checked signal-by-signal – stall, flush, forward_a/b – against hand-computed expected values.





KEY RESULTS

- Single-cycle and pipelined cores produce identical, correct results on the same programs.
- A real, silent correctness bug (ADDI misread as SUB) found and fixed – not just a syntax error.
- Every hazard class – data, load-use, control – resolved and independently verified in simulation.

<https://github.com/jatinsharma-tech>

<https://www.linkedin.com/in/jatin-sharma-a47595234>

jatinsharma04077@gmail.com

